claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_095035a0-e9f\agent-a4a649f3c9bc787ab.jsonl`
- SHA-256: `b80368471904e7c76966d3b868bb507074c2f7bdc00a49fc7e39bb0969a3edc3`
- Source modified: `2026-06-13T19:14:49+00:00`
- Imported at: `2026-07-08T16:00:00+00:00`
- project: `wf_095035a0-e9f`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-13T19:13:53.213Z)

Adversarially VERIFY this fusion-P2 review finding by reading the actual code. Try to REFUTE it; only confirm real=true if it genuinely holds against the code. Default to real=false if uncertain or if it's a nitpick/false-positive.

FINDING: {"title":"rally_gate.gate_windows recomputes the play-axis/net estimate over the WHOLE trajectory once per candidate window (O(W*P log P))","severity":"low","file":"backend/pipeline/segmenters/fusion_hybrid.py:77 (and rally_gate.py:91-105)","detail":"_enrich_candidate_stats calls rally_gate.gate_windows(points, [(cw['start'],cw['end'])], fps, min_crossings=0) once per window (driven by the per-window list comprehension in trajectory_hybrid.py:203-206). gate_windows internally calls estimate_play_axis(points) every call, which sorts ALL visible xs and ys (two _spread() sorts, each O(P log P)) and _median over all points, plus _points_in scans all P points ? even though axis/net are window-independent and explicitly documented as 'estimated once over the whole trajectory (stable)'. So for W windows over a P-point trajectory this is O(W * P log P) of redundant work. It is pure-Python and GPU-free, so not catastrophic, but wasteful on long videos with many candidates.","fix":"Estimate axis/net/deadband once per video (call estimate_play_axis(points) in the segmenter or precompute), then either count crossings per window directly via count_net_crossings, or call gate_windows(points, all_windows, ...) a single time and index the result by window ? instead of one whole-trajectory re-estimate per window."}

Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. The change implements fusion P2 (MULTI_SIGNAL_FUSION_PLAN.md): a new default-OFF `FusionSegmenter` (backend/pipeline/segmenters/fusion_hybrid.py) that subclasses TrajectoryHybridSegmenter, persists shuttle net_crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in backend/pipeline/detectors/fusion_features.py. NEW files: backend/pipeline/detectors/fusion_features.py, backend/pipeline/segmenters/fusion_hybrid.py. EDITED files (use `git --no-pager diff -- <path>` to see edits): backend/pipeline/segmenters/trajectory_hybrid.py, backend/pipeline/detectors/person_detector.py, backend/config/models.py, backend/pipeline/segmenters/__init__.py. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:13:56.046Z)

I'll verify this finding by reading the actual code. Let me start by examining the relevant files.

3. user (2026-06-13T19:13:57.011Z)

```
1       """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3       Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4       TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
```

```
5       do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6       shuttle-seeded window via two overridable seams the engine exposes:
7
8       - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9         GPU-free, computed from the trajectory we already have); and, only when the person cue
10        is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11        (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
12        fusion (P3). The person path lazily builds ONE `PersonDetector` ? so the default path
13        never imports torch / touches the GPU.
14      - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
15        (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
16        dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
17        **Persisted candidates remain the full superset** regardless, so the offline
experiment
18        harness can re-score any variant.
19
20      Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
21      nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
22      and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
23      segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
24      experiment harness (EXPERIMENT_HARNESS.md), never silently.
25      """
26      from typing import Any, Dict, List, Optional, Tuple
27
28      from backend.pipeline.segmenters.base import segmenter_registry
29      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
30      from backend.pipeline.detectors.base import DetectorRunner
31      from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
32      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
33      from backend.pipeline.detectors import rally_gate
34      from backend.pipeline.detectors import fusion_features as ff
35
36
37      class FusionSegmenter(TrajectoryHybridSegmenter):
38          """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
39
40          family = "fusion"
41          display_label = "Fusion"
42
43          def __init__(self, db: Any, config: Any):
44              super().__init__(db, config)
45              # Built lazily only if the person cue is enabled (keeps torch/GPU off the
default path).
46              self._person: Optional[Any] = None
47
48          @property
49          def name(self) -> str:
50              return "fusion_hybrid"
51
52          @property
53          def description(self) -> str:
54              return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
55                      "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
56                      "AI provider sets semantic boundaries.")
57
58          def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
```

```
Any]]:
            substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
            if substrate == "tracknet":
                cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
                return TrackNetRunner(cfg), {
                    "weights_path": cfg.weights_path,
                    "queue_length": cfg.queue_length,
                    "fusion_substrate": "tracknet",
                }
            wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
            return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}

        def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
                                    frame_width: float, video_path: str,
                                    idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
            stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
            # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
            # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
            # over the whole trajectory by rally_gate (camera-agnostic).
            gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
min_crossings=0)
            stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0

            fcfg = idx_cfg.get("fusion", {})
            if fcfg.get("cue_flags", {}).get("person", False):
                stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
            return stats

        def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
                             frame_width: float, video_path: str,
                             fcfg: Dict[str, Any]) -> Dict[str, float]:
            """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
            frame range and compute the scale/translation-invariant person features. Lazily
builds
            ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
            # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
            from backend.pipeline.detectors.person_detector import PersonDetector
            if self._person is None:
                self._person = PersonDetector(self.config)

            frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
            start_f, end_f = int(cw["start"] * fps), int(cw["end"] * fps)
            det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
            fw = det["frame_width"] or frame_width
            fh = det["frame_height"] or 0.0

            poly = fcfg.get("court_polygon")
            polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
            net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
            shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
            return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
                                              court_polygon=polygon, net=net)

        def _select_for_handoff(self, windows: List[Dict[str, Any]],
                                window_stats: List[Dict[str, Any]],
                                idx_cfg: Dict[str, Any]) -> List[int]:
            """Served Gate A/B over the (already-enriched) per-window stats. Default
```

```
thresholds
113             (0) keep every window ? identical to the substrate. Persisted candidates are
114             untouched (full superset)."""
115             fcfg = idx_cfg.get("fusion", {})
116             min_crossings = int(fcfg.get("min_crossings", 0))
117             min_players = int(fcfg.get("min_players", 0))
118             person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
119
120             keep: List[int] = []
121             for i, st in enumerate(window_stats):
122                 if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
123                     continue
124                 if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
125                     continue
126                 keep.append(i)
127             return keep
128
129
130     segmenter_registry.register(FusionSegmenter)
131
```

4. user (2026-06-13T19:13:57.313Z)

```
1       """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3       WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4       the opponent for service after losing a point, that single trajectory looks like a
5       rally. The discriminator is structural, not duration-based: a real rally has the
6       shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7       single service feed crosses ~once.
8
9       This module is camera-agnostic and needs NO new model ? it works on the trajectory
10      we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11      axis with the larger shuttle spread) and the net position (median shuttle coord on
12      that axis ? the shuttle clusters at the two ends and crosses the net region often,
13      so the median sits near the net), then counts sign changes of (coord - net) across
14      the visible points inside a candidate window, with a deadband to ignore jitter.
15
16      Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17      Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18      windows from trajectory_to_action_windows, or to fold into it later.
19      """
20      from __future__ import annotations
21
22      from dataclasses import dataclass
23      from typing import List, Sequence, Tuple
24
25      Interval = Tuple[float, float]
26
27
28      @dataclass
29      class GatedWindow:
30          start: float
31          end: float
32          crossings: int
33          visible_points: int
34          kept: bool
35
36
37      def _spread(vals: Sequence[float]) -> float:
38          """Robust spread = p95 - p5 (ignores a few outlier detections)."""
39          if len(vals) < 2:
40              return 0.0
41          s = sorted(vals)
```

```
        lo = s[max(0, int(0.05 * (len(s) - 1)))]
        hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
        return hi - lo


    def _median(vals: Sequence[float]) -> float:
        if not vals:
            return 0.0
        s = sorted(vals)
        n = len(s)
        return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])


    def estimate_play_axis(points) -> Tuple[str, float, float]:
        """Return (axis 'x'|'y', net_value, span) from visible points.

        Play axis = the image axis along which the shuttle travels most (larger spread).
        For a baseline camera that's Y (players top/bottom); for a side camera, X.
        """
        xs = [p.x for p in points if p.visible]
        ys = [p.y for p in points if p.visible]
        sx, sy = _spread(xs), _spread(ys)
        if sx >= sy:
            return "x", _median(xs), sx
        return "y", _median(ys), sy


    def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
        """Count sign changes of (coord - net) across visible points, ignoring the
        deadband zone near the net (so jitter at the net doesn't inflate the count)."""
        crossings = 0
        last_side = None
        for p in window_points:
            if not p.visible:
                continue
            v = (p.x if axis == "x" else p.y) - net
            if abs(v) < deadband:
                continue
            side = v > 0
            if last_side is not None and side != last_side:
                crossings += 1
            last_side = side
        return crossings


    def _points_in(points, fps: float, start: float, end: float):
        return [p for p in points if start * fps <= p.frame <= end * fps]


    def gate_windows(points, windows: List[Interval], fps: float,
                     min_crossings: int = 2, deadband_frac: float = 0.06) ->
List[GatedWindow]:
        """Annotate each (start,end) window with its shuttle net-crossing count and a
        keep decision (crossings >= min_crossings). Deadband is a fraction of the play
        axis span. Net/axis are estimated once over the whole trajectory (stable)."""
        axis, net, span = estimate_play_axis(points)
        deadband = deadband_frac * span
        out: List[GatedWindow] = []
        for (s, e) in windows:
            wp = _points_in(points, fps, s, e)
            c = count_net_crossings(wp, axis, net, deadband)
            vis = sum(1 for p in wp if p.visible)
            out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
                                   kept=c >= min_crossings))
        return out
```

```
106
107
108     def apply_gate(points, windows: List[Interval], fps: float,
109                    min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
110         """Convenience: return only the windows that pass the crossing gate."""
111         return [(g.start, g.end) for g in
112                 gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
113
```

5. assistant (2026-06-13T19:14:00.129Z)

Now let me look at the call site in trajectory_hybrid.py that drives the per-window enrichment, to verify the O(W*P log P) claim.

6. user (2026-06-13T19:14:01.565Z)

```
150             merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
151             min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
152             handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
153             ai_max_workers = idx_cfg.get("ai_max_workers", 5)
154             log_candidates = idx_cfg.get("log_yolo_candidates", True)
155             store_candidates = idx_cfg.get("store_yolo_candidates", True)
156
157             detection_params = {
158                 "detector": self.name,
159                 "velocity_thresh": velocity_thresh,
160                 "merge_gap": merge_gap,
161                 "min_action_window_duration": min_window_duration,
162                 **runner_params,
163             }
164
165             # --- Phase 1: env healthcheck (fail fast with a clear message) ---
166             ok, msg = runner.healthcheck()
167             if not ok:
168                 logger.error("%s WSL environment not ready: %s", label, msg)
169                 return []
170             logger.info("%s WSL env OK: %s", label, msg)
171
172             # --- Phase 2: trajectory -> candidate rally windows ---
173             # Trajectory detectors process the whole video in one pass (they carry
174             # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
175             t_start = time.time()
176             out_dir = os.path.join("output", self.family)
177             csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
178             if not csv_path:
179                 logger.error("%s produced no trajectory; aborting.", label)
180                 return []
181
182             points = runner.parse_trajectory_csv(csv_path)
183             logger.info("Parsed %d trajectory points (%d visible).",
184                         len(points), sum(1 for p in points if p.visible))
185
186             all_candidate_windows = runner.trajectory_to_action_windows(
187                 points, fps=fps, frame_width=frame_width, chunk_start=0.0,
188                 velocity_thresh=velocity_thresh, merge_gap=merge_gap,
189                 min_window_duration=min_window_duration, chunk_index=0,
190             )
191             logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
192                         label, time.time() - t_start, len(all_candidate_windows))
193
194             if log_candidates:
195                 for cw in all_candidate_windows:
196                     logger.info(f"[{label} rally]
{_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
197                                 f"({cw['end'] - cw['start']:.1f}s) |
```

```
frames={cw['active_frame_count']} "
                                f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")

            # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
            # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
            # persistence and the handoff gate below.
            window_stats = [
                self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
idx_cfg)
                for cw in all_candidate_windows
            ]

            # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
            candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
            if store_candidates:
                for i, cw in enumerate(all_candidate_windows):
                    candidate_ids[i] = self.db.add_candidate_segment(
                        video_id, detector=self.name,
                        start_time=cw["start"], end_time=cw["end"],
                        chunk_index=cw["chunk_index"], confidence=min(1.0,
cw["peak_velocity"]),
                        stats=window_stats[i], detection_params=detection_params,
                    )

            if not all_candidate_windows:
                return []

            # --- Phase 3: AI provider handoff over padded candidate clips ---
            # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
            # candidates above stay the full superset ? only the served play_segments are
gated.
            handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
            handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
            os.makedirs(handoff_dir, exist_ok=True)
            logger.info("Phase 3: %s validation on %d candidate windows...",
                        provider_name, len(handoff_indices))

            def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
                w_start = max(0, window["start"] - handoff_padding)
                w_end = min(video_duration, window["end"] + handoff_padding)
                w_dur = w_end - w_start
                w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
                if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
                    return []
                prompt = sport_profile.get_prompt(chunk_duration=w_dur)
                segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
                                                     label=f"Handoff {wi+1}")
                valid = []
                for seg in segments:
                    try:
                        seg["start_time"] = float(seg["start_time"]) + w_start
                        seg["end_time"] = float(seg["end_time"]) + w_start
                        seg["_candidate_id"] = candidate_ids[wi]
                        valid.append(seg)
                    except (ValueError, TypeError, KeyError) as e:
                        logger.warning("Dropping segment with unparseable timestamps: %s ?
%s", seg, e)
                try:
                    os.remove(w_path)
```

```
252                 except OSError:
253                     pass
254                 return valid
255 
256             ai_segments: List[dict] = []
257             with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
258                 futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
259                            for wi in handoff_indices]
260                 for future in futures:
261                     ai_segments.extend(future.result())
262             try:
263                 os.rmdir(handoff_dir)
264             except OSError:
265                 pass
266             logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
267 
268             # --- Phase 4: DB population + candidate linking ---
269             segments_data = []
```

7. assistant (2026-06-13T19:14:08.309Z)

Now let me confirm the claim by tracing the exact computational structure. Let me verify:

1. The list comprehension at line 203-206 calls `_enrich_candidate_stats` once per candidate window (W times).
2. Each call to `_enrich_candidate_stats` (fusion_hybrid.py:77) calls `rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps, min_crossings=0)` with a single-window list.
3. `gate_windows` (rally_gate.py:96) calls `estimate_play_axis(points)` over the FULL `points` list every time.
4. `estimate_play_axis` (rally_gate.py:61-66) does two list comprehensions over all points and two `_spread` calls (each sorts ? O(P log P)) plus `_median` (sorts again ? O(P log P)).

Let me verify there's no caching of axis/net anywhere and confirm `points` is the full trajectory passed unchanged.

8. user (2026-06-13T19:14:09.675Z)

```
backend\eval\distill_local.py:56:    gated = rally_gate.gate_windows(points, intervals, fps,
min_crossings=0)  # for net-crossings
backend\pipeline\detectors\rally_gate.py:55:def estimate_play_axis(points) -> Tuple[str, float,
float]:
backend\pipeline\detectors\rally_gate.py:91:def gate_windows(points, windows: List[Interval],
fps: float,
backend\pipeline\detectors\rally_gate.py:96:    axis, net, span = estimate_play_axis(points)
backend\pipeline\detectors\rally_gate.py:112:            gate_windows(points, windows, fps,
min_crossings, deadband_frac) if g.kept]
backend\pipeline\segmenters\fusion_hybrid.py:8:- ``_enrich_candidate_stats`` ? always adds the
**net-crossing count** (Gate A signal,
backend\pipeline\segmenters\fusion_hybrid.py:70:    def _enrich_candidate_stats(self, cw:
Dict[str, Any], points: List[Any], fps: float,
backend\pipeline\segmenters\fusion_hybrid.py:73:        stats =
super()._enrich_candidate_stats(cw, points, fps, frame_width, video_path, idx_cfg)
backend\pipeline\segmenters\fusion_hybrid.py:77:        gated = rally_gate.gate_windows(points,
[(cw["start"], cw["end"])], fps, min_crossings=0)
backend\pipeline\segmenters\trajectory_hybrid.py:82:    def _enrich_candidate_stats(self, cw:
Dict[str, Any], points: List[Any], fps: float,
backend\pipeline\segmenters\trajectory_hybrid.py:204:
self._enrich_candidate_stats(cw, points, fps, frame_width, video_path, idx_cfg)
```

9. assistant (2026-06-13T19:14:12.047Z)

The finding is confirmed. Let me also verify the base class `_enrich_candidate_stats` to ensure the `points` passed is the full trajectory (not a per-window subset) and that no axis caching exists.

10. user (2026-06-13T19:14:13.039Z)

```
82          def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
83                                      frame_width: float, video_path: str,
84                                      idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
85              """Stats dict persisted into ``candidate_segments.stats`` for one window. The
BASE
86              returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
87              rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
88              features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
89              return {
90                  "peak_velocity": cw["peak_velocity"],
91                  "mean_velocity": cw["mean_velocity"],
92                  "active_frame_count": cw["active_frame_count"],
93                  "track_id_count": cw["track_id_count"],
94              }
95
96          def _select_for_handoff(self, windows: List[Dict[str, Any]],
97                                  window_stats: List[Dict[str, Any]],
98                                  idx_cfg: Dict[str, Any]) -> List[int]:
99              """Indices of the candidate windows that proceed to the AI handoff (and thus may
100             become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
101             ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
102             Persisted candidates are NOT affected ? they remain the full superset for
offline
103             re-scoring."""
104             return list(range(len(windows)))
105
106         def process_video(self, video_id: str, video_path: str,  # type: ignore[override]
107                           player_pool: Optional[List[str]] = None,
108                           max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
109             # override-ignore: the ABC declares the Result contract (Success|Failure)
110             # but every live segmenter still returns a bare list ? the documented
111             # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
```

11. assistant (2026-06-13T19:14:29.093Z)

Confirmed. The docstring at line 88 explicitly states "`points` are the whole-video trajectory points", confirming the full trajectory is passed to each per-window call.

The finding holds completely. Let me trace through the exact computational claim one final time:

- **trajectory_hybrid.py:203-206** ? list comprehension over `all_candidate_windows` (W windows), each calling `_enrich_candidate_stats(cw, points, ...)` where `points` is the full whole-video trajectory.
- **fusion_hybrid.py:77** ? each call invokes `rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps, min_crossings=0)` with the full `points` and a single-element window list.
- **rally_gate.py:96** ? `gate_windows` calls `estimate_play_axis(points)` unconditionally at the top, over the full P-point trajectory.
- **rally_gate.py:61-66** ? `estimate_play_axis` builds two filtered lists over P points, then `_spread(xs)` + `_spread(ys)` (rally_gate.py:37-44, each sorts ? O(P log P)) plus two `_median` calls (rally_gate.py:47-52, each sorts ? O(P log P)).

So for W windows the axis/net estimate is recomputed W times over the full P-point trajectory: O(W · P log P). And the comment at rally_gate.py:95 explicitly documents axis/net as "estimated once over the whole trajectory (stable)" ? window-independent ? confirming the redundancy is real, not a per-window-dependent computation.

The severity assessment is fair: it's genuinely wasteful and redundant, but it is pure-Python,

GPU-free, and `FusionSegmenter` is default-OFF (opt-in, not the default segmenter). The per-window `_points_in` scan (rally_gate.py:88, O(P) over the full list, also redundant since it could index a sorted-by-frame structure) is unavoidable per-window work but is not the headline cost. The redundant part is specifically the `estimate_play_axis` re-estimation. Low severity is correct.

The fix suggestion is also valid and matches the code: estimate axis/net/deadband once per video, then either call `count_net_crossings` per window directly (the existing public function at rally_gate.py:69) or call `gate_windows(points, all_windows, ...)` a single time and index by window.

12. user (2026-06-13T19:14:44.325Z)

Structured output provided successfully

13. assistant (2026-06-13T19:14:49.615Z)

Verified the finding against the actual code and confirmed it as real (low severity). The redundant-work chain holds exactly as described: the per-window comprehension at trajectory_hybrid.py:203-206 calls into fusion_hybrid.py:77, which invokes `rally_gate.gate_windows` once per window with a singleton list, and that function unconditionally re-runs `estimate_play_axis(points)` (two O(P log P) sorts + a median sort) over the full trajectory on every call ? despite rally_gate.py:95 documenting axis/net as "estimated once over the whole trajectory (stable)." Result is O(W·P log P) of redundant work. Severity stays low because the path is pure-Python/GPU-free and `FusionSegmenter` is default-OFF. The structured output has been submitted.